Deep Learning Systems Analysis Report

Food Recognition with Convolutional Neural Networks

Report Overview

This project investigates 101-class food-image classification using convolutional neural networks implemented in PyTorch and trained from scratch. The main experiment increases channel width while holding other training settings fixed, improving test accuracy from 57.39% to 63.53%. Additional controlled experiments examine label smoothing, dropout, and an increase in maximum training epochs from 30 to 60. The wider CNN with dropout and the higher epoch limit achieves the best observed test accuracy of 65.63%, using the epoch-39 checkpoint selected by validation loss.

Dataset and Task Description

Food-101 contains 101,000 photographs covering 101 food categories. Its official partition provides 750 training and 250 manually reviewed test images per category; training images intentionally retain noise, including incorrect labels and intense colors (Bossard et al., 2014). The dataset supports supervised image classification through category labels and substantial variation in food appearance.

The dataset and its original documentation are available from the official Food-101 page hosted by ETH Zurich. The notebook downloads the dataset using torchvision.datasets.Food101.

The notebook obtains the data through torchvision.datasets.Food101. A stratified 80/20 split of the official training partition, using seed 42, produces 60,600 training and 15,150 validation images. Assertions confirm 600 training and 150 validation images per class, disjoint indices, and complete partition coverage. The official test set contains 25,250 images, or 250 per class.

Partition	Images	Images per class
Training	60,600	600
Validation	15,150	150
Test	25,250	250

Training uses random resized crops to 128 x 128 pixels, with crop area between 70% and 100% of the original and aspect ratio between 0.8 and 1.25, followed by horizontal flipping with probability 0.5. Validation and test resize the shorter side to 144 pixels and take a centered 128

x 128 crop. These choices limit computation while preserving varied training views, but may remove useful edge detail.

RGB statistics are calculated only from the training subset, using deterministic preprocessing and pixel-level accumulation in float64.

Channel	Mean	Standard Deviation
Red	0.55741	0.25712
Green	0.44290	0.26092
Blue	0.32874	0.26417

The same normalization is applied to every partition; no validation or test pixels contribute to these statistics.

Model Architecture and Design Decisions

The baseline is a compact convolutional neural network trained from scratch. It contains four convolutional blocks, each consisting of a 3 × 3 convolutional layer, batch normalization, ReLU activation, and 2 × 2 max pooling. The blocks produce 32, 64, 128, and 256 output channels, respectively.

Each convolutional layer learns filters that combine information across all incoming channels to produce feature maps. Padding of one pixel preserves spatial dimensions during convolution. Batch normalization normalizes intermediate activations and applies learned scaling and shifting, while ReLU introduces nonlinearity. Max pooling halves the height and width after each block, progressively reducing spatial resolution and computation (PyTorch Contributors, n.d.).

Stage	Baseline output per image	Wider CNN output per image
Input	3 × 128 × 128	3 × 128 × 128
Convolutional block 1	32 × 64 × 64	64 × 64 × 64
Convolutional block 2	64 × 32 × 32	128 × 32 × 32
Convolutional block 3	128 × 16 × 16	256 × 16 × 16
Convolutional block 4	256 × 8 × 8	512 × 8 × 8
Adaptive global average pooling	256 × 1 × 1	512 × 1 × 1
Flattening	256 values	512 values
Linear classifier	101 logits	101 logits

Note: Shapes are expressed as channels × height × width, excluding the batch dimension.

Adaptive global average pooling reduces each final feature map to one average value. Flattening then produces a feature vector for the linear classifier. This design keeps the classifier smaller than connecting every spatial position directly to the output classes and allows it to accept different final feature-map sizes (PyTorch Contributors, n.d.).

The classifier produces 101 logits, one unnormalized score for each food category. Softmax is not included in the model because PyTorch's cross-entropy loss accepts logits directly. The predicted category is the one with the highest logit (PyTorch Contributors, n.d.).

The baseline contains 414,853 trainable parameters. The wider CNN retains the same sequence of operations but doubles the channel widths to 64, 128, 256, and 512, increasing its parameter count to 1,603,749, approximately 3.87 times the baseline. This comparison tests whether greater feature capacity improves classification enough to justify the additional computation.

For the dropout experiments, a dropout layer with probability 0.3 is placed after flattening and before the linear classifier. It randomly suppresses features during training and is disabled during evaluation. This tests whether regularization improves the wider model's generalization without changing its convolutional blocks or trainable parameter count (Srivastava et al., 2014).

Experimental Comparison

Four controlled comparisons examined how channel width, label smoothing, dropout, and maximum training epochs affected food classification. Each comparison changed one aspect relative to its stated reference configuration.

Shared Experimental Settings

All configurations used the same training and validation split, image preprocessing, augmentation, batch size of 32, and random seed of 42. Models were trained from scratch using Adam with a learning rate of 0.001. Validation used un-smoothed cross-entropy for every configuration, providing a common basis for checkpoint selection.

Training stopped after five consecutive epochs without a lower validation loss, and the checkpoint with the lowest validation loss was restored for evaluation. The maximum was 30 epochs except in Experiment 4.

Experiment 1: Increasing Channel Width

This experiment compared the baseline CNN, with channel widths of **32, 64, 128, and 256**, against a wider CNN with widths of **64, 128, 256, and 512**. The number of blocks, layer types,

and remaining training settings were unchanged. Neither configuration used label smoothing or dropout.

The purpose was to investigate whether additional feature capacity improved classification enough to justify increased computation. Doubling the channel widths increased the trainable parameter count from **414,853 to 1,603,749**. This is the primary architectural comparison.

Experiment 2: Introducing Label Smoothing

This experiment compared the baseline CNN trained with standard cross-entropy against the same architecture trained with **label smoothing of 0.1**. Label smoothing mixes the hard class target with a uniform distribution, reducing the emphasis on assigning all probability to the labeled category (Szegedy et al., 2015).

The experiment investigated whether softened targets improved generalization, particularly given the documented noise in Food-101's training partition (Bossard et al., 2014). Architecture and other training settings remained unchanged. Validation and test losses were calculated without smoothing; training losses therefore represent different objectives and are not directly comparable.

Experiment 3: Adding Dropout to the Wider CNN

This experiment compared the wider CNN without dropout against the same model with **dropout probability 0.3**, applied after flattening and before the linear classifier. Label smoothing remained disabled in both configurations.

The wider CNN's increasing separation between training and validation accuracy motivated testing additional regularization. Dropout randomly suppresses features during training, discouraging excessive reliance on particular features (Srivastava et al., 2014). The experiment tested whether this improved validation performance while retaining the same convolutional architecture and trainable parameter count. The probability of 0.3 was a fixed exploratory choice rather than a tuned optimum.

Experiment 4: Increasing Maximum Training Epochs

This experiment compared the wider CNN with dropout under maximum limits of **30 and 60 epochs**. The earlier run reached its lowest validation loss at epoch 29, with almost identical loss at epoch 30, suggesting that further training might improve performance.

Only the maximum epoch count changed; early-stopping patience remained five epochs. The extended experiment started from scratch with the same seed because earlier checkpoints did not contain the optimizer and random states required for exact continuation. This comparison investigated whether the original epoch limit constrained learning while allowing early stopping to end training before epoch 60.

Comparison Structure and Evaluation Scope

Experiment	Reference configuration	Modified configuration	Single change
1	Baseline CNN	Wider CNN	Channel width
2	Baseline CNN	Baseline with label smoothing	Training-target smoothing
3	Wider CNN	Wider CNN with dropout	Dropout
4	Wider CNN with dropout, maximum 30 epochs	Same model, maximum 60 epochs	Maximum epochs

The experiments were developed sequentially. The wider-model, dropout, and extended-epoch experiments were introduced after earlier test results had been observed. Although all checkpoints were selected using validation loss, these later test evaluations should be interpreted as exploratory benchmark comparisons rather than independent confirmation on an untouched holdout.

Results and Interpretation

Overall Performance

Table 1 summarizes performance at the checkpoint selected by minimum validation cross-entropy for each configuration. Validation and test losses use un-smoothed cross-entropy throughout, allowing comparison across all configurations.

Table 1. Performance at the selected checkpoints.

Configuration	Selected epoch	Validation accuracy	Test accuracy	Test cross-entropy
Baseline CNN	30	52.62%	57.39%	1.6429
Baseline CNN with label smoothing 0.1	26	52.83%	58.21%	1.6891
Wider CNN	30	58.01%	63.53%	1.4201
Wider CNN with dropout 0.3, maximum 30 epochs	29	58.58%	63.70%	1.3887
Wider CNN with dropout 0.3, maximum 60 epochs	39	60.20%	65.63%	1.3240

Increasing channel width produced the largest improvement from a single change. Dropout provided a smaller additional benefit, while
increasing the maximum epochs allowed the dropout model to improve further. Label smoothing increased accuracy slightly but also increased
cross-entropy, demonstrating a trade-off between the two metrics.

Experiment 1: Increasing Channel Width

Doubling the channel widths increased test accuracy from 57.39% to 63.53%, a gain of 6.14 percentage points, or 1,550 additional correct
predictions among 25,250 test images. Test cross-entropy decreased from 1.6429 to 1.4201. Both metrics therefore support the wider CNN's
improvement in this run.

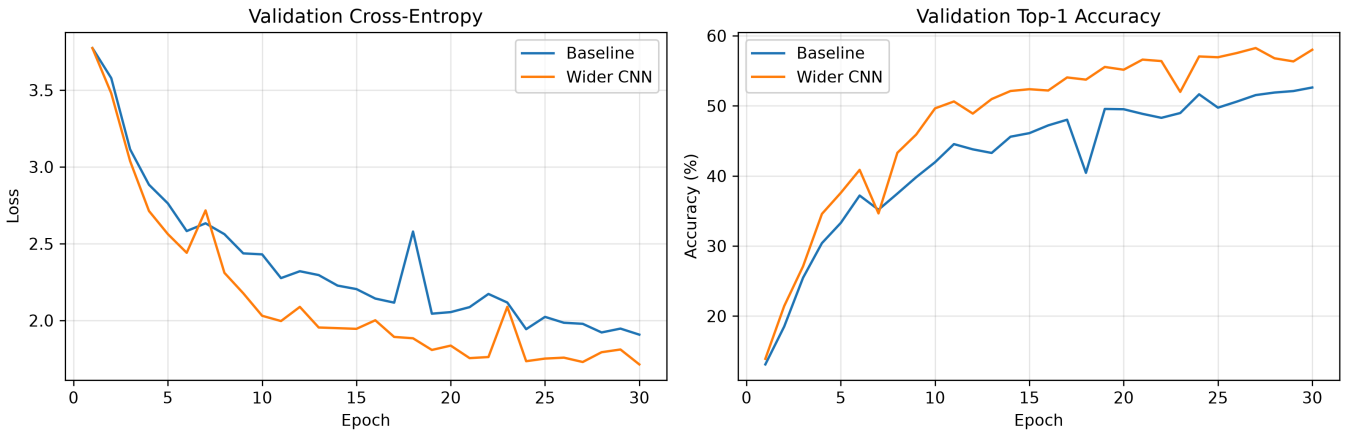


Figure 1. Validation cross-entropy and top-1 accuracy for the baseline and wider CNN. Both configurations use a maximum of 30 epochs,
without dropout or label smoothing.



Figure 2. Training and validation cross-entropy for the baseline and wider CNN, shown on the same vertical scale. Neither configuration uses dropout or label smoothing.

Training loss decreases steadily for both models, while validation loss declines less smoothly. At epoch 30, training and validation losses are 1.5948 and 1.9080 for the baseline, compared with 1.0312 and 1.7141 for the wider CNN. The wider model fits the training data more closely and achieves lower validation loss, but also exhibits a larger separation between the curves. This suggests greater overfitting risk rather than sustained validation deterioration, since its lowest validation loss occurs at epoch 30.

Training loss is measured during parameter updates with random image augmentation, whereas validation loss evaluates the end-of-epoch model using deterministic preprocessing. The separation therefore reflects differences in evaluation conditions as well as generalization.

The improvement required additional computation. Trainable parameters increased from 414,853 to 1,603,749, approximately 3.87 times as many. A typical training-and-validation epoch increased from approximately 2.4 to 3.5 minutes, around 46% longer on the recorded Apple Silicon MPS environment. These measurements describe epoch runtime, not inference latency.

Experiment 2: Introducing Label Smoothing

Label smoothing increased baseline test accuracy from 57.39% to 58.21%, a gain of 0.82 percentage points. However, test cross-entropy increased from 1.6429 to 1.6891, so the change did not improve both evaluation metrics.

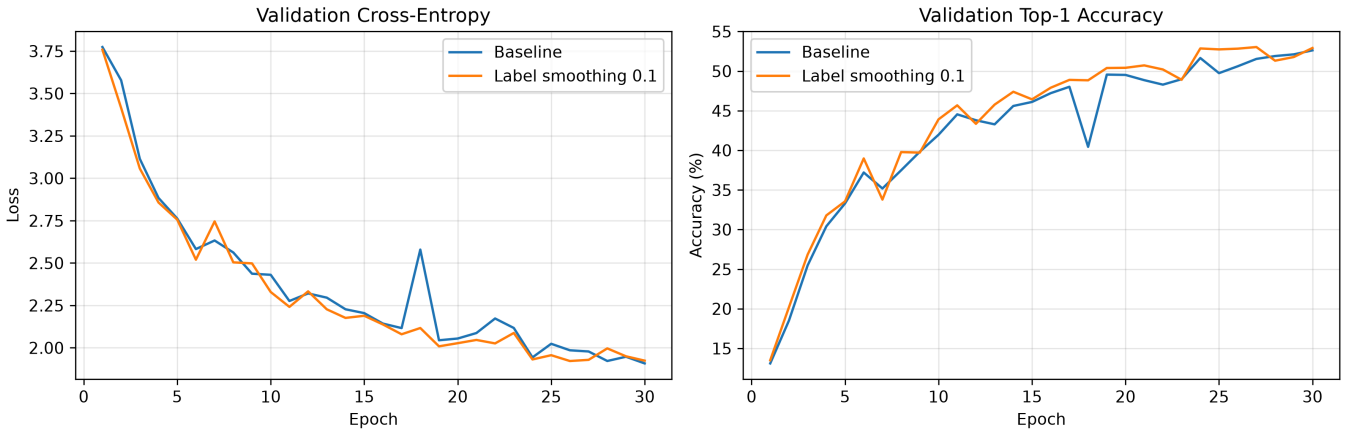


Figure 3. Validation cross-entropy and top-1 accuracy for the baseline CNN with and without label smoothing of 0.1. Validation cross-entropy is calculated without smoothing for both configurations.

The validation curves show broadly similar learning trajectories. At the selected checkpoints, label smoothing achieved slightly higher validation accuracy but higher validation loss, matching the direction of the test-set trade-off. The result does not establish that label smoothing consistently improves this architecture, nor does it establish better probability calibration, which was not measured.

Experiment 3: Adding Dropout to the Wider CNN

Adding dropout reduced validation cross-entropy from 1.7141 to 1.6580 and increased validation accuracy from 58.01% to 58.58% at the selected checkpoints. Test cross-entropy decreased from 1.4201 to 1.3887, while test accuracy increased from 63.53% to 63.70%, corresponding to
42 additional correct predictions.

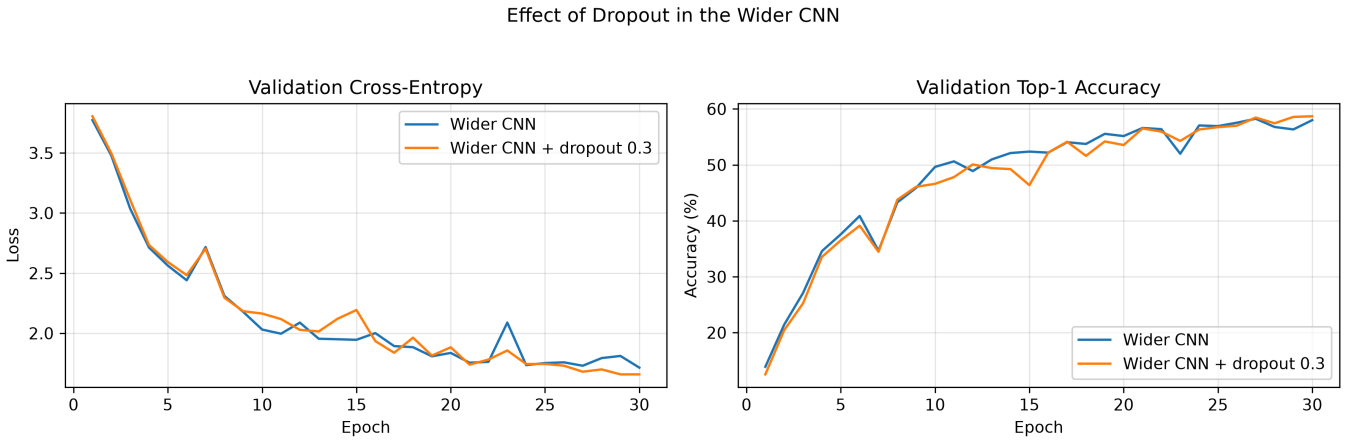


Figure 4. Validation cross-entropy and top-1 accuracy for the wider CNN with and without dropout of 0.3. Both configurations use a maximum of 30 epochs and no label smoothing.

These results suggest a modest benefit for this run, with a clearer improvement in cross-entropy than in top-1 accuracy. The selected dropout checkpoint came from epoch 29, rather than epoch 30, because it had slightly lower validation loss. Dropout is active when training accuracy is recorded but disabled during validation, so a smaller training–validation accuracy gap alone would not demonstrate reduced overfitting.

Experiment 4: Increasing Maximum Training Epochs

Increasing the dropout model's maximum epochs from 30 to 60 improved selected-checkpoint validation accuracy from 58.58% to 60.20% and reduced validation cross-entropy from 1.6580 to 1.5948. Test accuracy increased from 63.70% to 65.63%, a gain of 1.93 percentage points, or 488 additional correct predictions. Test cross-entropy decreased from 1.3887 to 1.3240.

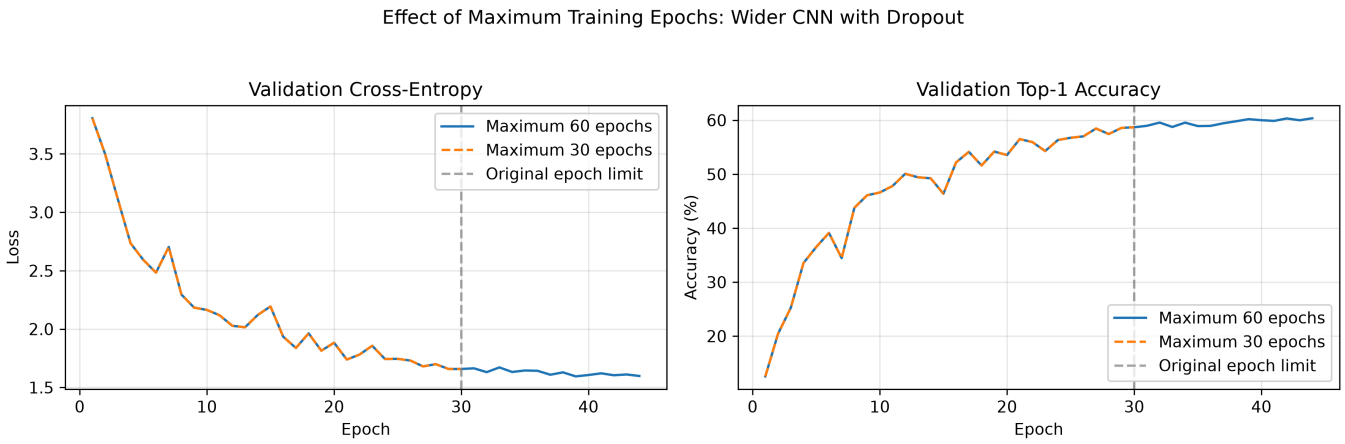


Figure 5. Validation performance for the wider CNN with dropout under maximum limits of 30 and 60 epochs. The curves overlap through epoch 30 to the recorded precision. The dashed line marks the original 30-epoch limit; the extended run ends at epoch 44.

Early stopping ended the extended run at epoch 44 after five consecutive epochs without improving on the validation loss achieved at epoch 39. The epoch-39 checkpoint was restored for test evaluation. Although epoch 44 had slightly higher validation accuracy, checkpoint selection consistently followed minimum validation loss.

The improvement indicates that the original 30-epoch limit constrained performance for this configuration. It does not imply that accuracy would continue improving indefinitely with additional epochs.

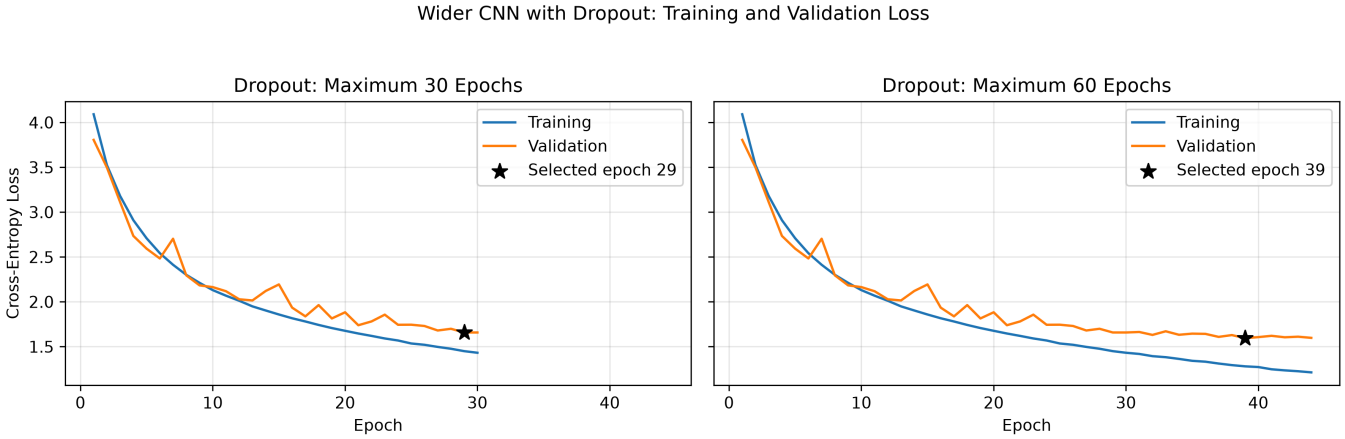


Figure 6. Training and validation cross-entropy for the wider CNN with dropout under maximum limits of 30 and 60 epochs. Stars identify the checkpoints selected by minimum validation loss. The extended run stopped at epoch 44.

Training loss continued to decrease beyond epoch 30, while validation loss improved more gradually and reached its minimum at epoch 39. During epochs 40–44, training loss continued decreasing without a new minimum validation loss, triggering early stopping. This illustrates why lower training loss alone is insufficient for checkpoint selection.

Training uses random augmentation and active dropout, whereas validation uses deterministic preprocessing with dropout disabled. Consequently, the distance between the curves is not a pure measure of overfitting.

Class-Level Error Analysis

The following analysis examines the wider CNN without dropout, which achieved 63.53% test accuracy. It does not describe the final extended dropout model.

Per-class recall varied substantially, from 19.6% for apple pie to 96.8% for edamame. Other low-recall categories included chocolate mousse (24.8%), pork chop (26.4%), and steak (28.0%). High-recall categories included hot-and-sour soup (90.4%) and miso soup (89.6%). Each category contained 250 test images, so unequal test class counts do not explain these differences.

Table 2. Most frequent directional confusions for the wider CNN.

True category	Predicted category	Number of errors	Percentage of true category
Hamburger	Pulled pork sandwich	67	26.8%
Steak	Filet mignon	64	25.6%
Samosa	Spring rolls	42	16.8%
Fried rice	Risotto	38	15.2%

These confusions suggest difficulty distinguishing foods with similar appearances. For example, steak and filet mignon can share visual characteristics, while rice-based dishes may have overlapping textures. The confusion counts identify recurring errors but do not establish which features caused the predictions.

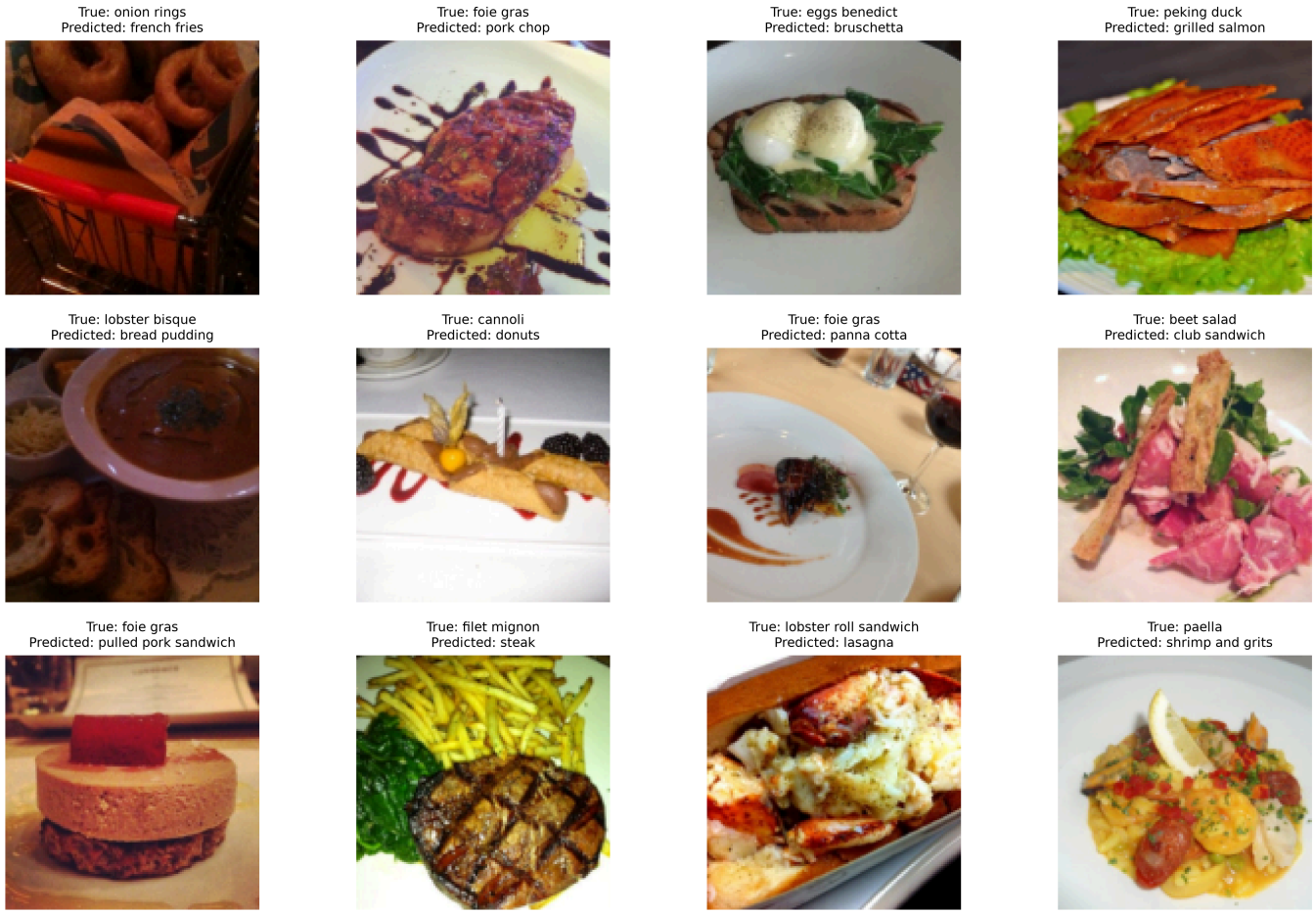


Figure 7. Twelve reproducibly sampled misclassified test images from the wider CNN without dropout. Normalization is reversed for display, showing the resized and cropped inputs seen by the model.

The sampled errors show visually related dishes, varied presentation, and challenging lighting. Filet mignon was predicted as steak, while eggs Benedict served on bread was predicted as bruschetta. Three foie-gras images differed substantially in appearance and received different incorrect predictions. These observations suggest plausible sources of difficulty but do not prove that cropping, background content, or particular textures caused an error. Because the sample contains only misclassifications, it does not represent all test images.

Interpretation Scope

The strongest observed improvement came from increasing channel width, followed by extending training for the wider CNN with dropout. The best observed configuration achieved 65.63% test accuracy, compared with 57.39% for the baseline, an overall gain of 8.24 percentage points.

All results come from one training seed and one validation partition; small differences should therefore be interpreted cautiously. The wider-model, dropout, and extended-epoch

experiments were introduced after earlier test results had been observed. Although checkpoint selection always used validation loss, the later test evaluations are exploratory benchmark comparisons rather than independent confirmation on an untouched holdout.

Limitations and Risks

The experiments use one seed and one validation partition. No repeated-run uncertainty estimates or paired significance tests are provided, so small accuracy differences should be treated as descriptive. The 128 x 128 inputs may suppress fine detail, and center cropping can omit food components. A compact four-block CNN trained from scratch is also a restricted architectural search.

The test set was first evaluated after the baseline and label-smoothing runs. Width, dropout, and budget extensions were added after those results were visible. Their checkpoints still use validation loss, but the overall development process was not insulated from test feedback; later test scores are exploratory benchmark results rather than an independent, untouched confirmation.

Validation is drawn from the noisy official training partition, whereas official test labels were manually reviewed (Bossard et al., 2014). This difference is a plausible contributor to consistently higher test accuracy, but the project does not measure its causal effect. Lower validation performance should not be interpreted solely as a sampling-size issue.

Training accuracy is measured during parameter updates with augmentation, and dropout runs additionally disable random features. Comparing its gap with validation across configurations therefore does not isolate overfitting. The notebook records performance and histories, but earlier checkpoints do not include optimizer state, random states, or a completion marker, limiting exact interrupted-run recovery.

Ethical and Responsible Use

The 101-category catalog does not establish equal coverage of cuisines, preparation styles, or user populations. Poor recognition of dishes outside that catalog could create unequal usefulness; the closed-set classifier must still choose a known category. Deployment would require evaluation on the intended food and user distribution, along with an explicit way to abstain on unfamiliar inputs.

A food-category prediction cannot establish ingredients, allergens, portion size, or nutritional content. The observed confusions make it unsuitable as a sole basis for allergy or medical dietary decisions. Any demonstration should describe predictions as uncertain image classifications and require ingredient verification rather than infer safety from the predicted category. These are application-specific risk assessments, not harms measured in this experiment.

Future Improvements

First, repeat the main comparisons across several seeds and quantify variability using paired predictions. Reserve a new external holdout before additional model development. Next, evaluate higher resolution, a learning-rate schedule, or transfer learning in separate comparisons, selecting settings with validation data rather than chasing test scores. Repeat class-level error analysis for the final model and assess unfamiliar-food rejection before considering deployment.

The reproducibility package should include the notebook, requirements.txt, dataset access instructions, saved split indices, histories, and checkpoints for fast evaluation. The recorded environment uses Python 3.14.0, PyTorch 2.14.1, and torchvision 0.29.1 on arm64 MPS. Pinned packages improve traceability but do not guarantee identical installation or numerical results on other platforms.

References

Bossard, L., Guillaumin, M., & Van Gool, L. (2014). Food-101 - Mining discriminative components with random forests. In *Computer Vision - ECCV 2014* (pp. 446-461). Springer. https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/

PyTorch Contributors. (n.d.). *torch.nn* [Software documentation]. PyTorch 2.14 documentation. Retrieved October 15, 2026, from https://docs.pytorch.org/docs/2.14/nn.html

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(56), 1929-1958. https://www.jmlr.org/papers/v15/srivastava14a.html

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2015). *Rethinking the Inception architecture for computer vision* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1512.00567